Exp. 1

Q1. Wap to declare a class student having data member as name, Roll.no accept & display data for one student.

```
#include <iostream>
using namespace std;
class student{
    string name;
    int roll-no;
public:
    void accept(){
    cout << "Enter student's name:";
    getline(cin,name);
    count << "Enter roll numer:";
    cin >> roll-no;
    }
    void display(){
    cout << "\n student details:\n";
    cout << "Name:" << name << endl;
    cout << "Roll Number:" << roll-no << endl
    }
};
int main(){
    student S;
    s.accept();
    S.display
    return 0;
}
```

Q. 2. WAP to declare a class book having data members as. ~~name, p~~ id, name, price. accept data for 2 book & display data of book having greater price

```
#include <iostream>
using namespace std;
class Book{
private:
    int id;
    string name;
    float price;
public:
    void Accept(){
        cout << "Enter book ID : ";
        cin >> id;
        cin.ignore();
        cout << "Enter book name : ";
        getline(cin, name);
        cout << "Enter book price : ";
        cin >> price
    }
    void display(){
        cout << "\n Book ID " << id << endl;
        cout << " Book Name " name << endl;
        cout << "Book price : " << price << endl;
    }
    float getprice(){
        return price;
    }
};
int main(){
```

```
Book b1, b2;
cout << "Enter details for Book 1:\n";
b1.accept();

cout << "\n Enter details for Book 2:\n";
b2.accept();

cout << "\n Book width greater price:\n";
if (b1.getPrice() > b2.getPrice()){
    b1.display();
} else {
    cout << "Both books have the same
                price.\n";
    b1.display();
    b2.display();
}
return
```

Page No.: YOUVA
Date:

Q.3 WAP to declare a class Time having data member as H, M & S Accept data for one object & display total time in seconds.

```
#include<iostream>
using namespace std;
class Time {
private:
  int H, M, S
public:
  void getTime(){
   cout << "Enter Hours:";
   cin >> H;
   cout<<"Enter Minutes:";
   cin >> M;
   cout<<"Enter seconds:";
   cin >> S;
}
int toSeconds (){
   return (H*3600)+(M*60)+S;
}
void displayTotalSeconds(){
    cout <<"Total time in seconds :"
        << toSeconds()<<endl;
}
int main(){
  Time t;
  t.getTime();
  t.displayTotalSeconds();
  return 0;
}
```

Exp. 2

Q.1. WAP to declare class 'students' having data members as name and population. Accept this data for 5 cities and display name ~~of~~ of city having highest population

```
#include<iostream>
  using namespace std;
 class city {
   public:
       int p;
       string name;
       void accept(){
       cout<<"Enter city name & popular:";
       cin>>name>>p;
     }
       void disp(){
       count<<"City with the highest
            population is:"<<name
     "with population "<<p<<endl;
     }
   };
  int main(){
      city c[5];
      int maxIndex = 0;
      for (int i=0; i<5; i++){
      c[i].accept();
    }
   for (int i=1; i<5; i++){
   if (c[i].p>c[maxIndex].p){
     maxIndex = i;+
  }}
}
```

Q.

```
#include <iostream>
using namespace std;
class Account{
public:
int account_no;
float balance;
void accept(){
 cout<<"Enter account numbe & balance
 cin >> account_no >> balance
}
 void applyInterest (){
  if ( balance >=5000) {
      balance += (balance * 0.10);
  }
 }
 void display (){
  cout <<" Account Number : " << account_
  no <<", Balance after interest :"
  << balance << endl;
 }
};
 int main(){
  Account accounts[10];
  for( int i = 0; i <10; i++) {
   accounts [i] . accept();
  }
```

```
cout << "\n Accounts with balance
for (int i = 0; i < 10; i++){
 account[i].accept();
 if (accounts[i].balance >= 5000) {
   accounts[i].display();
  }
}
 return 0;
}
```

// output.

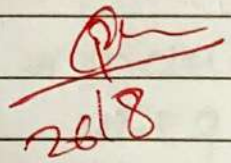
Enter acc number and balance: 1 / 599
Enter acc number and balance: 2 / 799
3 / 999
4 / 899
5 / 699
6 / 499
7 / 399
8 / 299
9 / 299
10 / 100

Acc with balance >= 5000 after applying int

c. WAP to declare a class 'staff' having data members as name and post. Accept this data for 5 staff and display names of staff who are "HOD"

```
#include<iostream>
 using namespace std;
 class staff
 {
  public;
  string name, post;
  void accept()
  {
   cout<<"Enter name of staff";
   cin>>name;
   cout<<"Enter post of staff";
   cin>>post;
  }
  void display()
 {
 cout<<name<<endl;
  }
 };
 int main()
 {
  staff s[5];
  for(int i=0;i<5;i++)
  {
   s[i].accept
  }
 count<<" The name of staff who are
       HOD are: "<<endl;
```

```
for (int i = 0; i < 5; i++)
{
  if(s[i].post == "HOD")
    s[i].display();
  }
  return 0;
}
```



Exp. 3.

Q.1. Write program to declare a class 'book containing data member as book. title author name and price. Accept and display information for one object using a pointer to that object.

```
#include <iostream>
using namespace std;
class book
{
public;
string btitle, atitle;
int price;
void accept()
{
cout<<"Enter name of book";
cin >> b.title;
cout<<" Enter the name of author";
cin >> title;
cout<<"Enter the price of the book";
cin >> price;
}
void disp()
{
cout<<"The name of the book is"
    <<title<<endl;
cout<<"The name of the author is"
    <<atitle<<endl;
cout<<"The price of the book is"
    <<price<<endl;
}
}
```

```
{ int main ()
  {
   book b;
   book p;
   p = &b;
   p -> accept ();
   p -> disp ();
   return 0;
  }


// Output
Enter name of book : xyz
Enter name of author: abc
Enter price of book: 99
The name of the book is xyz
The name of the author is abc
The price of the book is 99
```

Q.2 WAP to declare class student having data members roll_no, and percentage. using 'this' pointer invoke member function to accept and display this data for one obj.

```
#include <iostream>
 using namespace std;
 class student
 {
  public:
  int roll; per;
  vod accept ()
  {
   cout << "Enter the roll no"
    cin >> this → roll;
   cout << " Enter the percentage";
    cin >> this → per;
  }
   void display ()
   {
    this → accept ();
    cout << " The roll.no is <<
       this → roll << endl;
    cout << " The percentage is " <<
      this → per << " % " << endl;
     }
   };
int main ()
  {
   student s;
    s1. display()
    return 0
  }
```

// Output:
Enter roll no. 68
Enter percentage 46
The roll no is 68
The percentage is @ 46%.

Q.3 Write a program to demostrate use of nested class

```
#include<iostream>
using namespace std;
class student {
int roll; string name, roll;
 void accept()
 {
  cout<<"Enter name of student:";
  cin>>name;
  cout<<"Enter roll number of student:";
  cin>>roll;
 }
void display()
{
  cout<<"The name of student is "
    <<name<<"\n";
```

```
    cout << "The roll no. of student is:"
         << roll << "\n";
  }
};
class marks
{
  public:
   int m1, m2
   void accept(){
    cout << Enter marks of first sub: \n;
    cin >> m1;
    cout << Enter marks of second sub: \n;
    cin >> m2;
}
void display(){
float per = ((m1 + m2)/200.0) * 100;
cout << "The percentage of two sub
     is:" << per << "%.d" << endl;
  }
};
student s;
marks m;
s.accept();
m.accept();
s.display();
m.display();

return 0;
}
```

26/8

Date: YOUVA

Experiment 4

Q1. WAP to swap

```
#include <iostream>
using namespace std;
class number{
private:
int a, b;
public:
int temp;
void accept(){
cout<<"Enter the first number";
cin>>b;
}
void disp(){
cout<<"\n After swapping:"<<endl;
cout<<"First number ="<<a<<endl;
cout<<"Second number ="<<d;
}
void swap (numbers &n){
n.temp = n.a;
n.a = n.b;
n.b = n.temp;
}
};
int main(){
numbers n;
n.accept();
n.swap();
```

```
n.disp();
return 0;
}
```

// Output:

```
Enter the first number : 9
Enter the second number: 2

After swapping:
First number = 2
Second number = 9.
```

Q2. WAP to swap two number from same class using concept of friend function.

```
# include <iostream>
  using namespace std;
  class neumber{
   private:
   int a, b, temp;
   public;
   void accept(){
    cout << "Enter the first numbers":
    cin >> a;
    cout << " Enter the second number";
    cin >> b;
   }
   void disp(){
    cout << " In After swapping :" << endl;
    cout << " First number =" << a << endl;
    cout << " 'Second number -" << b;
   }
```

```
    friend void swap (number &);
};
void swap(number &n){
        n.temp= n.a;
        n.a = n.b;
        n.b = n.temp;
}
int main (){
        number n;
        n.accept();
        swap(n);
        n.disp();
        return 0;
}
```

Output:

```
Enter the first number :7
Enter the second number:4
After swapping
first number =4
second number =7
```

Q.3 WAP to swap two number from different class using friend function

```
#include <iostream>
using namespace std;
class num2;
class num1 {
  private:
  int a;
  public:
  void accept() {
   cout << "Enter the first number:";
   cin >> a;
  }
  friend void swap(num1 &, num2 &);
};
class num2 {
  private:
  int b;
  public:
  void accept() {
  cout << "Enter second number:";
  cin >> b;
  }
  friend void swap(num1 &, num2 &);
};
void swap (num1 &x, num2 &y) {
 int temp;
 temp = x.a;
 x.a = y.b;
 y.b = temp;
 cout << "\n After swapping:" << endl;
 cout << "first number=" << x.a << endl;
 cout << "second number=" << y.b << endl;
```

```
}
int main (){
    num1 n1;
    num2 n2;
    n1.accept();
    n2.accept();
    swap(n1, n2);
    return 0;
}
```

// Output.

```
Enter first number=5
Enter second number=6.
After swaping:
first number =6
first Second number =5
```

Q4. WAP to create two classes Result1 and Result2 which stores the marks of student Read the value of marks for both class obj. and compute the average of two result

```
#include <iostream>
    using namespace std;
    class result2;
    class result1 {
            private;
            string name;
            float marks
          public:
```

```
void accept(){
   cout << "Enter student name:";
   getline(cin, name);
   cout << "Enter the total mark
    obtain in first semester out of 100.
   cin >> marks;
  }
  friend void average( result1, result2);
};
class result2{
     private:
     float marks;
     public:
     void accept(){
     cout << "Enter total marks obtained
      in second semester out of 100:";
     cin >> marks;
     }
     friend void average (result1, result 2);
  };
void average(result x, result2 y){
    float avg;
    avg = (x.marks + y.marks)/2;
    cout << "\n Average of both the
            results = " << avg.
  }
  int main(){
    result1 r1;
    result2 r2;
     r1.accept();
     r2. accept();
     average(r1, r2);
  return 0;
  }
```

Output:
Enter student name : Swaraj Chandanshiv
Enter total marks obtain in first semester out of 100 : 98
Enter total marks obtained in seond semester out of 100 : 96
Average of both results = 97.

5. Swap two numbers from different class using friend function.

```
#include <iostream>
using namespace std;
class num2;
class num1
{ Public:
  int n1;
  void accept()
  {
   cout << "Enter first number:";
   cin >> n1;
  }
 void display()
 {
  cout << "Num1:" << n1 << endl;
 }
 friend void swap(Num1 & nu1, Num2 &
 };                              nu2);
class Num2
{
 Public:
   int n2;
```

```
void accept()
{
cout<< "Enter second number:";
cin>> n2;
}
void display()
{
cout<<"Num 2"<<n2<<endl;
}
```

6. Create 2 classes, class A and class B each with private. Write a friend function Sum() that can access private data from both class and return the sum

```
#include<iostream>
using namespace std;
class B;
class A{
private:
int a;
pubilic:
voied accept()
{
cout<<"Enter first number:"
cin>>a;
}
friend void sum(A,B);
};
class B;
{
private:
int b;
public:
void accept()
{
cout<<"Enter the second number:";
cin>>d;
}
friend void sum(A,B);
};
```

```
void sum (A x, B y)
{
 int sum;
 sum = x.a + y.b;
 cout << "\n Sum of the 2 number = " << sum;
}
 int main ()
{
 A h1;
 B h2;
 h1.accept ();
 h2. accept ();
 sum (h1, h2);
}
```

// Ourput

Enter first number : 5

Enter second number : 6

Sum of the 2 number = 11

7. WAP with a class Number that contains a private integer. use friend function swap Numbers(Number , , Number) to swap private value of 2 Num object.

```
#include <iostream>
 using namespace std;
 class Number
{
 private;
 int a, b;
 public:
 void accept()
{
 cout << "Enter the first no:";
 cin >> a;
 }
 void accept ()
 {
 cout << "Enter second no:";
 cin >> b;
 }
 void display()
{
 cout << "\n After Swapping :" <<
 cout << " First number =" << a << endl;
}
 void display()
{
 cout << "Second number " << b;
 friend void swap (Number & x, Number &y
};
```

```
void swap ( Number & x, Number & y)
 {
  int temp;
  temp = x.a;
   x.a = y.b;
   y.b = temp;
 }
 int main()
 {
   Number n1;
   Number n2;
   N1. accept();
   N2.accept();
   swap (n1, n2);
   n1. display ();
   n2. display();
   return 0;
  }
```

→ output:

```
Enter the first number : 7
Enter the second number : 4
After swapping
First Number = 4
Second Number = 7
```

8. Define 2 classes Box and Cube, each having a private volume, Write a friend function find. greater (Box, Cube) that determines which object has a larger value

```
#include <iostream>
 using namespace std;
 class cube.
 {
  privare:
   float volume;
  public:
  void accept()
 {
  cout << "Enter volume of a cube";
  cin >> volume;
 }
  friend void findgrt (cube, box);
  ;
};
class Box.
{
 private:
  float vol;
  public:
  void accept()
 {
  cout << "Enter the volume of box:";
  cin >> vol;
 }
```

```
friend void findgrt (cube, Box);
};
void findgrt (cube x, box y)
{
  if (x.volume > y.vol)
  {
    cout << "\n cube has longer volume";
  }
  else {
  cout<<"\n boobx has longer volume";
    }
}
int main()
{
  cube c;
  box b;
   c.accept();
   b.accept();
   findgrt(c,b);
   return 0;
   }.
```

→ Output

```
Enter the volume of cube : 567.88
Enter the volume of box : 985.66
Box has larger volume.
```

9. Create a class complex with real and imaginary parts as privare members use a friend function to add 2 complex number and return the result as a new complex object

```
#include<iostream>
 using namespace std;
 class complex.
 {
  privare:
   int r, i;
  public:
  void accept()
  {
  cout<<" Enter the real part:";
  cin>>r;
  cout<<" Enter the imaginary part;
  cin>>i;
  }
   void disp()
 {
  cout<<r<<"+"<<"i"<<endl;
 }
  friend complex sum(complex, coplex);
};
 complex sum(complex x, complex y)
 complex temp;
 temp.r = x.r + y.r;
 temp.i = x.i + y.i;
 return temp;
}
```

```
int main()
{
complex c1, c2, c3;
cout<<" Enter first complex number:\n";
c1.accept()
cout << "Enter the second complex
    number:\n";
c2.accept()
cout << "\n # complex Number1 \n";
c1.disp()
cout << " # complex Number2 \n";
c2.disp()
c3 = sum(c1, c2)
cout << "\n sum of the 2. complex
    numbers =";
c3.disp();
return 0;
}
```

// Output:

```
Enter the first complex number:
Enter the real part:5
Enter the imaginary part:6.
```

18. Create class point with private member x and y. Write friend function that calculate and return the distance between 2 point object.

→
```
#include <iostream>
# include namespace std
  namespace s
  class point.
 {
  int x, y;
  public:
  void accept()
 {
  Cout << "Enter the x. co.ordinate:";
  cin >> x;
  cout <<"Enter the y co.ordinare:
  cin>>y;
 }
  void disp()
 {
  cout << " (" << x << ", " << y << " )" << en
 }
  friend double diff (point, point);
 }
  double diff ( point x, Point y)
 {
  double temp;
  return temp = Sqrt ( power (x.x - y.x);2)
     + pow((x.y - y.y), 2));
 }
 int main ()
 {
```

```
point p1, p2;
cout << " For the point P : \n";
p1. accept();
cout << " For the point Q : \n";
p2. accept();
cout << " P = ";
p1. disp()
cout << " Q ";
p2. disp();
cout " << " \n Difference between the
    2 points = << diff (p1; p2) << "unit
}
```

Output :-

```
For the point P:
Enter the x. co.ordinate : 5
Enter the y. co.ordinate : 6
For the point Q :
Enter the x. co.ordinate : 7
Enter the y. cordinate : 8
P=(5,6)
Q = (7.8).
```

10. Create class student with private data me
name and three sub marks. Write friend fu
calculate Avg (Student) that calculates.

```
#include<iostream>
using namespace std;
class student;
{
string name;
int math;
int phy;
int chem;
public:
void accept()
{
cout<<"Enter student name:";
getline(cin, name);
cout<<"Enter marks obtained in math";
cin>>math;
cout<<"Enter marks obtained in physic";
cin>>phy;
cout<<"Enter marks obtained in chem:";
cin>>chem;
}
friend void calculate Average (student);
};
void calculate average (Student x)
{
float avg;
avg = (x.math + x.phy + x.chem) / 3;
cout<<"\n Average marks = "<<avg;
}
```

```
int main()
{
 Student S;
 S.accept();
 calculate average(S);
 return 0;
}
// Output
Enter student name: Swaraj
Enter mark in math: 99
Enter marks in physic: 97
Enter marks in chem: 98
```

11. Create 3 classes: Alpha, Bera, gamma each with a private data member write single friend function that each access all three and print their sum.

```
# include <iostream>
  using namespace std;
  class alpha
  {
    int a;
    public;
    void accept()
  {
   cout << " Enter the First number :"
   cin >> a;
  }
   friend void sum (Alpha, Bera, Gamma)
};
```

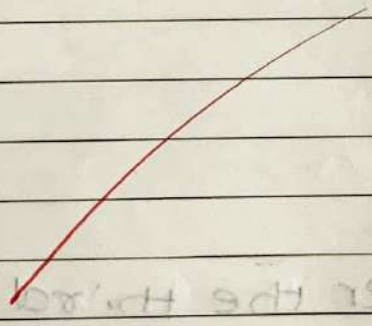
4/11

```
class Bera
{
 int b;
 public:
 void accept()
 {
  cout << "Enter second number:";
  cin >> b;
 }
  friend void sum (Alpha, Bera, Gama);
};
 class Gamma,
 {
   int q;
  public:
  void accept()
  {
   cout << "Enter the third number:";
   cin >> q;
  }
   Friend void sum (Alpha, Bera, Gumma);
   };
   void sum (Alpha x, Bera y, Gumma z
   {
    int Sum;
    Sum = x.a + y.b + z.q;
    cout << "In Sum = " << Sum;
   }
    int main ()
   {
    Alpha a;
    Bera b;
    Gamma q;
```

```
a. accept();
b. accept();
q. accept();
sum(a, b, q);
 return 0;
}
```

// Output

Enter the first number : 5:
Enter the second number : 3
Enter the third number : 9
Sum = 18



13. Create 2 classes: BankAccount and Audit. BankAccount holds private balance information Write a friend function in Audit that access and print balance information for auditing.

```
#include<iostream>
using namespace std;
class Audit;
class BankAccount
{
 private:
  float balance;
 public:
  void accept()
  {
   cout<<"Enter the balance information
   cin>>balance;
  }
  friend class Audit;
 ;
}class Audit
 public;
 void disp(BankAccount x)
 {
 cout<<"Balance information="
       <<x.balance;
 }
};
int main()
```

```
{
Bank Account b;
Audit a;
b.accept();
a.disp(b);
return 0;
}.
```

Experiment 5

Q.1

```
#include <iostream>
using namespace std;

class SumCalculator{
private:
    int n, sum;
public:
    SumCalculator(int num): n(num), sum(0)
    {
        for (int i=1; i<=n; ++i)
        sum += i;
    }
    void display() { cout << "Sum from 1 to"
    << n << " is" << sum << endl; }
};
int main() {
    SumCalculator calc(10);
    calc.display();
    return 0;
```

Date:

8.

// Default constructor

```
# include <iostream>
using namespace std;
class Sum{
private:
   int n;
public:
  Sum() {
      n = 10;
      cout << "Default Constructor: n is
  set to" << n << endl;}
int calculareSum(){
   int sum = 0;
   for(int i=1 ; i<= n; i++){
   sum += i;
  }
  return sum;
  }
};
int main(){
  Sum sum1;
  cout << "Sum of num from 1 to" << 10 << "
" is" << sum1.calculareSum
  return 0;
}
```

Date: YOUVA

// Parameterized constructor.

```
#include<iostream>
using namespace std;
class Sum{
private:
  int n;
public:
  Sum(int x){
    n=x;
    cout<<"parameterized constructor
  n is set to"<<n<<endl;
}
  int calculateSum(){
  int sum=0
  for(int i=1; i<=n; i++){
    sum+=i;
  }
  return sum;
}};
int main(){
  int n;
  cout<<" Enter a value for n:";
  cin>>n;
  Sum sum2(n);
  cout<<"Sum of numbers from 1 to "<<n<<
  is "<<sum2.calculateSum()<<endl;
  return 0;
}
```

Q.2 WAP to declare class student having data member as name and percentage

```
#include<iostream>
#include<string>
using namespace std;
class student
{
private:
string name;
float percentage;
public:
student (string n, float p)
{ name=n
  percentage=p;
  cout<< Name:
  getline (cin, name)
  cout<<"Percentage";
   cin>> percentage
 }
  void display()
{ cout<<" Student name:"<<name<<endl;
  cout<<" Student percentage: "<<percenta
                              <<endl;
 }
int main()
{ student S1 (~~string~~"Swaraj", 81.5)
  S1. display()
  return 0;
 }
```

// Output
Name: ~~a~~Swaraj
Percentage: 81

Q.3 Define a class 'College' member var

```
#include<iostream>
#include<string>
using namespace std;
class college{
private:
  int roll_no;
  string name;
  string course;
public:
college(int r, string n, string c=
  "Computer Engineering"){
roll_no =r;
name = n;
course = c;
}
void display(){
cout<<"Roll No :"<<roll_no<<endl;
cout<<"Name:"<<name<<endl;
cout<<"Course :"<<course<<endl;
}
};
int main(){
int roll_no1, roll_no2;
string name1, name2, course1, course2;
cout<<"Enter roll no, Name and course
for first student:"<<endl;
cout<<"Roll_no:";
```

```
cin >> roll_no1;
cin.ignore()
cout << "Name:";
getline(cin, name1);
cout << "\n Enter roll no, Name and course
 for second student:" << endl;
cout << " Roll NO:";
cin >> roll_no2;
cin.ignore();
cout << "Name:";
 get line(cin, name2);
 cout << "Course (Press Enter for default):";
get line (cin, course2);

college studen1(roll_no1, name1, course1.empt
() ? "compter Engineering" : course1);
college student2(roll_no 2, name2, courseD.
empty() ? "compurer Engineering": course2);

cout << "\n Student 1 Dare" << endl;
Student1.display();
cout << "\n Student 2 Dare" << endl;
Student2.display();

return 0;
}
```

Q.4. Write a programe to demonstrate virtual base class. Assume suitable data with figu

```
#include <iostream>
using namespace std;
class rectangle
{
  int l, b;
 public :
 rectangle ()
 { l=2;
   b=5;
 }
  rectangle (int x)
 {
  l = x;
  b = x;
 }
 rectangle [int x, int y]
 {
   l=x;
   b=y;
 }
 rectangle (rectangle &r3)
 {
  l=r3.l;
  b=r3.b;
 }
  void calculate ()
 {
  cout << " The area is :" << (l * b) << endl;
  }
};
```

```
int main()
{
 rectangle r1;
  r1 calculate()
  rectangle r2(3).
  r2 .calculate();
  rectangle - r3(6,6);
  r3 . calculate()
 rectangle r4 (r3);
  r4. calculate();
  return 0;
}
```

Output-

The area is : 10
The area is : 9
The area is : 36
The area is : 36

Experiment 6.

a) ~~Write a program to implement multi level inheritence. Assume suitable data~~

```
#include <iostream>
using namespace std;
class person
{ protected:
  string name;
  int age;
  public:
  void accept[]{
  cout<<"Enter name:";
  cin>>name;
  cout<<"Enter age:";
  cin>>age;
  }
  void display(){
  cout<<"Name:"<<name<<endl;
  cout<<"Age :"<<age<<endl;
  }
};
class student : public person{
private:
  int rollNumber;
public:
  void accept Student Details(){
  accept();
  cout<<"Enter roll number:")
  cin>>roll (number;
}
```

```
void displayStudentDetails(){
accept();
cout<<"Roll Number :"<<rollNumber<<endl;
}
};
int main(){
Student s;
s.acceptStudentDetails();
s.displayStudentDetails();
return 0;
}
```

Q 2. Create two base class academic and sports for multilever inheritance

- Academic class - Marks of student
- Sports class - score
- Derived class - Result inherite from both classes and total score

```
#include <iostream>
using namespace std;
class Academic{
public;
 int marks;
 void getmarks(){
 cout<<" Enter academic marks:";
 cin>>marks;
 }
};
class sports{
 public:
  int score;
  void getscore(){
  cout<<" Enter sport score:";
  cin>>score;
 }
};
class result: public academic, public
sports {
public:
void display(){
```

```
int total = marks + score;
cout << "\n Academic marks:" << marks;
cout << "\n Sports score:" << score;
cout << "\n Total score :" << total << endl;
}};

int main(){
Result R;
R.getmarks();
R.getscore();
R.display();
return 0;
};
```

Q.3 Create a class vehicels with attributes like brand and mobel derive a class from vehicle which add an attribute type further derive a class electric car from car which add battery capacity.

```
#include <iostream>
#include <string>
using namespace std;
class vehicle{
public:
 string brand, model;
};
class car : public vehicle{
public:
  string types;
};
class Electrican : public car{
public:
  int battery capacity;
  void input(){
   cout << "Enter brand:";
   cin >> brand;
   Cout << " Enter model :";
   cin >> model;
   cout << " Enter type :";
   cin >> type;
  cout << " Enter battery capacity :";
  cin >> battery capacity
  }
```

```
void display(){
cout << "\n -- car details --";
cout << " Brand :" << brand << endl;
cout << " Model :" << model << endl;
cout << " type :" << type << endl
cout << " Battery capacity :" << battery
    capacity << endl;
}
};
int main(){
electriciane;
e.input();
e.display();
return 0;
}
```

8 set opr
9. joint sqv
10 strin functi

Q.4. Hierarchical Inheritance

```
# include < iostream>
  using namespace std;
  class empoyee{
  public:
  int emp id;
  string name;
  void set data (int id, string n) {
  empID = id;
  name = n;
  }
  void display (){
  cout << "Employe ID:" << empID << endl;
  cout << "Name :" << name << endl;
  }
};
  class manager : public employe {
  public;
  string department
  void set department (string dept){
  department = dept;
  }
  void display (){
  employe :: display ()
  cout << "Department :" << department
  << endl;
  }
};
  class developer : public employee
{
```

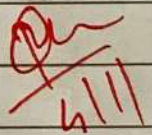
```
public:
string programing langer
void setprograming long (string long) {
programming long = rang;
}
void display (){
Employee :: display()
cout << "Programing language." << programin
long << endl;
}
};
int main (){
manager M;
M.set Data (101, "Swaraj");
M.set Department ("CEO");
Developer d;
d.set Data (102, "Gk Athorva");
d.set programing long ("Python");
cout << " Manager Details" << endl;
M.display ();
cout<<"In Developer Derails << endl;
d.display ();
return 0;
}
```

Q5. Hybrid & inheritance

```
#include <iostream>
using namespace std;
class person {
public;
string name;
  int age;
  void setperson(){
  ~~en~~ame = n;
   age = a;
  }
  void display person(){
  cout << "Name:" << name << endl;
  cout << "Age" << age << endl;
  }
};
class student : public person
{
 public;
  int student ID;
  void setstudent (int ID) {
  student ID = ID;
 }
 void display student () {
 cout << "Student ID:" << student ID
  << end;
 }
};
```

```
class sports
{
 public;
  int sportsscore;
  void setscore (int score){
  sports score = score;
 }
  void display score()
  {
  cout <<" Sports score: " << sportsscore;
  }
};
class result : public student, public sport
{
 public;
  int marks;
  void setmarks (int m){
  marks = n;
  }
   void se int calculate total (){
   return marks + sportsscore;
  }
   void display result (){
   display person ();
   display student();
   display sports()
   cout << "Marks" << marks << endl;
   cout<< "Total (marked + sports score);
    " << calculate Total << endl;
```

```
};
int main(){
Result R;
R.serperson("Bob", 35);
R.setstudent(102);
R.setscore(45);
R.setmarks(90);
R.displayresult();
return 0;
}
```



// Experiment 7

a. WAP using function overloading to calculate the area of a laboratory (which's rectangu in shap) & area of class room (which is square in shape)

```
#include <iostream>
 using namespace std;
 class area;
{
 private:
 float l, b;
 public:
 void area(float l)
{
 float area;
 area = l * l;
 cout << "Area of square:" << area<< endl;
 }
 void area (float l , float b)
 {
  float area;
  area = l * b;
  cout << "area of rectangle:" << area;
 }}
 int main()
 {
  area a1;
  a1.area(8);
  a1.area(4,12);
  return 0;
 }
```

b) WAP using function overloading to calculate the sum of 5 float value & sum of 10 integer value

```
#include <iostream>
using namespace std;
class sum
{ private :
  int a, b, c, d, e, f, g, h, i, j;
  float k, l, m, n, o;
  public;
  void sum (float k, float l, float m, float n,
     float o)
  {
   float = sum;
   sum = k + l + m + n + o;
   cout << "Sum of 5 floating num" <<sum<<
  }
  void sum (int a, int b, int c, int d, int e
     int f, int g, int h, int i, int j)
  { int sum;
    sum = a + b + c + d + e + f + g + h + i + j;
    cout << "Sum of of 10 integers" << sum;
    }
};
int main ()
{ sum s1;
  s1 . sum( 1 . 2 , 3.5, 5.6, 8.4, 1.5);
  s1 . sum (1, 2. 3, 4, 5, 6, 7, 8, 9, 10); 9y
  return 0;
}
```

Experiment - 8.

1 WAP to overload '+' operator so that two string can be concatenate eg: "xyz" + "par" = xyz par.

```
#include <iostream>
#include <string>
using namespace std;
class MyString{
public: string str;
MyString operator + (MyString & other){
temp str = str + other str;
return temp;
}
void display()
{
  cout << str << endl;
}
};
int main()
{
MyString S1, S2, S3;
S1. Str = "xyz";
S2. Str = "par";
S3 = S1 + S2;
cout << "Concatenated string;
S3. display();
return 0;
}
```

b. WAP to create base class 'login having data m
name & password. Declare accept() function
virtual. Derive Emaillogin and membership logi

```
#include <string>
#include <iostream>
 using namespace std;
 class login {
  protected: string name, password;
  public: virtual void accept(){
          cout << "Enter name:";
          cin >> name;
          cout << "password ";
          cin >> password;
      }
  }
 class emaillogin :Public login{
    string email;
  public:
    void accept() override{
     login :: accept()
    Cout <<"Enter email:";
    Cin >> email;
  }
 void display(){
  cout << "\n---- Email login Details---\n"
  Cout << "Name:" << name << "password:" <<
  password << "Email" << email << endl;
 };
```

```
class Membership login : Public login{
    string membership ip;
  public :
    void accept () override{
    login : accept ();
    cout << " Enter membership id ";
    cin >> membership id ;
   }
    void display () {
   cout <<"\n ---Membership login Det--
   cout << " Name" << name << "\n pasword <<
   pasword << "membership id " << member
  }
 }
 int main()
{
  Email login e;
  Membership login m;
  e.accept();
  m. accept ();
  e. display ();
  m. display ();
   return 0;
}
```

4/11

Experiment. 9

a) Write a program to copy the contents of one file into another. One "firstr.text" in read (ios::in) mode and "second txt" file in write (ios::out) mode. Copy the contents of "first.txt" into "second.tx Assume "first.txt" is already created

```
# include <iostream>
# include <fstream>
 using namespace std;
 int main(){
   fstream first_file;
   first_file.open("first text", ios::in);
   if(!first_file).{
    cout << "error" << endl;
     return 1;
   }
   while (first file.get(en){
    second_file.put(ch); }
   first_file.does();
   second_file.close(); }
 cout << "file copied successfully!" << endl;
  return 0;
}
```

b) Write a c++ program to count Digits and Spaces using file Handing.

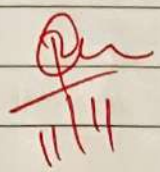
```
#include <iostream>
#include <fstream>
#include <ctype>
using namespace std;
int main () {
  fstream new-file;
  new-file.open ("First.text",ios::in);
  if (! new-file){
  cout << "error" <<
   return 1;
  }
 int digits-count =0;
 int space_count = 0;
 int char ch; while (new_file.get(ch)){
   if(is digit (ch)){
     digits -count ++;
   }
 if( is space (ch)){
   space_count ++;
 }
 new_file.close ();
 cout << " No. of digits" << digits_count <<endl;
 cout <<" No. of space : " << space-count<<en
  return 0;
}
```

c) WAP to count words using file handling?

```
#include <iostream>
#include <fstream>
#include <ecrype>
using namespare std;}

 fstream new-file;
 new-file.open ("first.rexf", ios::in);
 (! new_file) {
  cout <<" inter occured !" <<endl;
  return 1;
 }
 char ch}
 int words -count = 0;
 bool inword = false:
 while (new-fils gets (ch)){
  if (ispace (ch)){
   inlord = false.}
  else if(! inword)
   {
    words -count ++;
     int word = true:}
   }}
 }
 new file.close ():
 cout <<"No.of words:" <<words.count<< endl;

 return 0;
}
```

d)

```
#include <iostream>
# include < fstream>
# include <string>
  using namespace std;

  std int main(){
   ifstream file("first txt.");
   if(!file){
     cout << " Error" << endl;
     return 1;
   }
    string word, target;
      int count = 0;
   cout << "Enter word to count " << endl;
   cin >> target;

   while (file >> word){
    if (word = target)
      count ++ ++;
   }
   cout <<" occurence of " << target << " ":
   << cout << endl;
   file.close ();
  }
```



Experiment. 10

a. Write a C++ program to find sum of array elements using function template (eg. Pass float and double array of 10 elements)

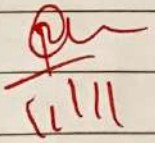
```
#include <iostream>
using namespace std;

template <class T>
T sumArray(T arr[], int n){
T sum = 0
 for (int i = 0; i<n ; i++)
 sum + = arr[i];
 return sum;
}
 int main(){
 int arr(5) = { 1, 2, 3, 4, 5};
 float flo-arr(5) = { 1.1, 2.2, 3.3, 4.4, 5.5};
 double dec_arr[5] = { 0.5, 1.5, 2.5, 3.5, 4.5};

cout << "Sum of integer array"
 <<sum Array (int_arr, 5) <<endl;
cout << "Sum of float array" << sum Array
 (flo-arr, 5) << endl;
cout <<" Sum of Double array:"
 <<Sum Array (dec-arr, 5) <<endl;

 return 0;
}
```

~~Exp. 10.~~

```
#include <iostream>
#include <string>
  using namespace std;

 template <typenameT>
 T square (T num){
    return num * num;
 }
  template <>
  string square <string>(string str){
   return str + str;
  }
   int main(){
   int num = 5;
   double dec_num = 2.5;
   string str = "Apple";

 cout <<"Square of integer"<< square(num);
 cout <<" Square of double" <<squore (dec_num);
 cout <<"Square of string:" <<square(str) <<endl

  return 0;
}
```

```
#include<iostream>
#include<math.h>
using namespace std;

template<class T1, class T2> class calc{
public:
T1 x;
T2 y;
calc(T1 n, T2 m){
 x=n;
 y=m;
}
 void sum(){
 cout<<"Sum"<<x+y<<endl;
}
 void pro(){
 cout<<"Product:"<<x*y<<endl;
}
 void diff(){
 cout<<"Difference"<<x-y<<endl;
}
 void quo(){
 cout<<"Quotient"<<x/y<<endl;
}
 void rem(){
 cout<<"Remainder:"<<x%y<<endl;
}
 void power(){
 cout<<"x raised to y is:"<<power(x,y)<<endl
```

```
void min_num(){
  cout << min of number :" <<fmin(x,y)<<endl;
}
void max_num(){
  cout << max of number "<<fmax (x,y)<<endl;
}
  void sin_x (){
 cout <<"sin of x: "<<sin (x) <<endl;
}
  void cos_y(){
  cout <<"cos of y :" << cos(y)<<endl;
}
};
  main (){
  calc < int, int > n (100, 200);

  n.sum();
  n. diff();
  n. pro ();
  n. quo();
  n. rem()
  n. power()
  n. min_num()
  n. max_num();
   n. sin_x()
    n. cos_y();
}
```



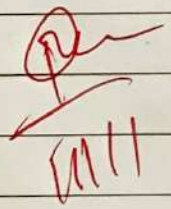
// Without iterator.

```
#include <iostream>
#include <vector>
#include <ctype>
using namespace std;
int main(){
vector char> v(10); unsigned int i;
cout << "size = " << v.size() << endl;
for(i=0; i<10; i++){ v[i] = i+'0';
cout << " current content : \n";
for (i=0; i<10; i++){ v.push_back(i>10+
 cout << "size now=" << v.size() << endl;
 coul-<< " current content:\n".
for (i=0; i< v.size(); i++){ cout<< v[i]<<"
 cout << "\n\n";
for (i=0; i< N.size(); i++){ u[i]= to upper
 cout<<"modified contents:\n"; [vi]);}
for (i=0; i< N.size(); i++){ cout<<v[i]<<" ";
 return 0;
}
```

// With iterators

```
#include <iostream>
#include <vector
using namespace std;
int main(){
```

```
vector <char> v(10);
vector<char> :: iterator p;
int i;
p = v.begin();
i = 0;
while(p != v.end()){
  *p = i + 'a';
   p++;
   i++;
}
cout << " original content : \n";
p = v.begin();
while(p != v.end()){
 cout << *p << " ";
  p++;
}
```



Experiment 12

Q. WAP using STL

```
#include<iostream>
#include<stack>
#include<queue
using namespace std;
int main(){
stack<int>S;
S.push(10);
S.push(20);
S.push(30);

cout<<"Stack top element :"<<S.top()<<endl;
S.pop();
cout<<"After pop, top element :<<stop()<<en
cout<<"Stack elements";
while(!S.empty()){
cout<<S.top()<<" ";
S.pop();
}
cout<<endl;
cout<<"\n (b) queue implem

queue<int> q;
q.push(100);
q.push(200);
q.push(300);
```

```
cout << "Front element " << q.front() << endl;
cout << "Back element " << q.back() << endl;
q.pop();
cout << "After pop, new front element
        << q.front() << endl;
cout << "Queue elements: ";
    while (!q.empty()){
  cout << q.front() << " ";
   q.pop();
}
  cout << endl;
  return 0;
}
```